Indian Institute of Technology Jodhpur

CSL 7640: Natural Language Understanding

---

# Assignment-2

---

**Submitted to:** Prof. Anand Mishra

**Name:** Saher Dev

**Roll No:** B23CS1059

**Date:** March 25, 2026

# Contents


**I Problem 1: Word Embeddings from IIT Jodhpur Data** **3**

**1 Introduction** **3**

**2 Task 1: Dataset Preparation** **3**
2.1 Data Sources . . . 3
2.2 Preprocessing Pipeline . . . 3
2.3 Dataset Statistics . . . 4
2.4 Word Cloud . . . 4

**3 Task 2: Model Training** **5**
3.1 Architecture Overview . . . 5
3.1.1 Skip-gram with Negative Sampling . . . 6
3.1.2 CBOW with Negative Sampling . . . 6
3.1.3 Negative Sampling Loss (Custom Implementation) . . . 6
3.2 Sliding Window & Training Pair Generation . . . 6
3.3 Baseline Training . . . 7
3.4 Hyperparameter Experiments . . . 7
3.4.1 Analysis of Hyperparameter Effects . . . 8

**4 Task 3: Semantic Analysis** **8**
4.1 Nearest Neighbors (Top 5) . . . 9
4.1.1 Interpretation: Why Are Similarities So High? . . . 9
4.2 Word Analogy Experiments . . . 10
4.2.1 Discussion . . . 10

**5 Task 4: Visualization** **11**
5.1 t-SNE Projections . . . 11
5.1.1 Interpretation . . . 11
5.2 PCA Projection . . . 12
5.3 Cosine Similarity Heatmap . . . 13
5.4 Summary of Visualizations . . . 13

**6 Conclusion** **14**

**II Problem 2: Character-Level Name Generation Using RNN Variants** **15**

**7 Task 0: Dataset Generation** **15**

**8 Task 1: Model Implementation** **15**
8.1 Model 1: Vanilla RNN . . . 16
8.2 Model 2: Bidirectional LSTM (BLSTM) . . . 16
8.3 Model 3: RNN with Bahdanau Attention . . . 18
8.4 Hyperparameters and Their Significance . . . 20

**9 Task 2: Quantitative Evaluation** **21**
9.1 Training Loss Curves . . . . . 22

**10 Task 3: Qualitative Analysis** **23**
10.1 Failure Modes . . . . . 23
10.2 Realism of Generated Names . . . . . 24
10.3 Representative Generated Samples . . . . . 24

**11 Conclusion** **25**

# Part I
# Problem 1: Word Embeddings from IIT Jodhpur Data

## 1 Introduction

Word embeddings are dense, low-dimensional vector representations of words that capture semantic and syntactic relationships from co-occurrence patterns in text. This report documents the end-to-end pipeline for training **Word2Vec** models—both **Continuous Bag of Words (CBOW)** and **Skip-gram with Negative Sampling**—on a domain-specific corpus scraped from IIT Jodhpur web pages.

The report is organized into four tasks: (1) Dataset Preparation, (2) Model Training with hyperparameter experiments, (3) Semantic Analysis via nearest-neighbor retrieval and word analogies, and (4) Visualization of the learned embedding space using t-SNE, PCA, and cosine-similarity heatmaps.

## 2 Task 1: Dataset Preparation

### 2.1 Data Sources

Textual data was collected from the following four IIT Jodhpur web pages:

1. **CSE Department Page** – https://www.iitj.ac.in/computer-science-engineering
2. **CSE Faculty Page** – https://www.iitj.ac.in/computer-science-engineering/en/faculty
3. **List of Academic Programs** – https://iitj.ac.in/office-of-academics/en/list-of-academic-programs
4. **Academic Regulations** (mandatory) – https://iitj.ac.in/office-of-academics/en/academic-regulations

HTML was fetched via the `requests` library and parsed with `BeautifulSoup`. Boilerplate elements (`<script>`, `<style>`, `<header>`, `<footer>`) were stripped before extracting visible text.

### 2.2 Preprocessing Pipeline

The following preprocessing steps were applied sequentially to the raw extracted text:

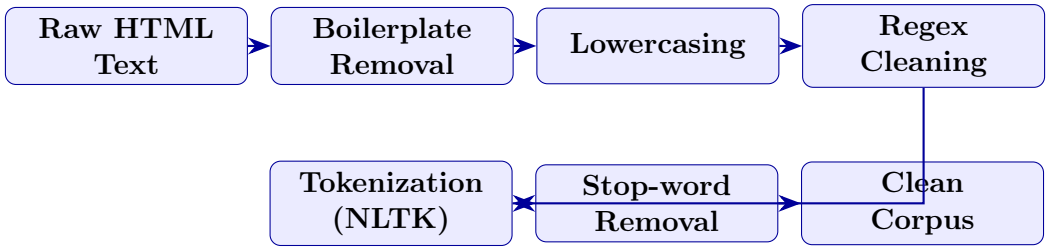



Figure 1: End-to-end text preprocessing pipeline.

(i) **Boilerplate Removal:** BeautifulSoup strips HTML tags, scripts, styles, headers, and footers, retaining only visible page text.

(ii) **Lowercasing:** All text is converted to lowercase to ensure case-insensitive matching.

(iii) **Punctuation & Non-textual Removal:** A regex filter `[^a-z\s]` removes all characters that are not English letters or whitespace, effectively discarding numbers, special symbols, and non-English text.

(iv) **Tokenization:** The cleaned string is tokenized using `nltk.word_tokenize`.

(v) **Stop-word Removal:** Standard English stop-words from `nltk.corpus.stopwords` are filtered out. Additionally, tokens of length $\leq 1$ are discarded to remove residual noise.

**Why Stop-word Removal Matters**

Without stop-word filtering, ultra-frequent grammatical words such as *the*, *and*, *to*, and *for* dominate the co-occurrence statistics. Their overwhelming presence compresses all content words into a narrow region of the vector space, preventing the model from learning meaningful semantic distinctions among academic terms. By stripping these out, we allow the embedding space to allocate capacity to domain-relevant relationships.

## 2.3 Dataset Statistics

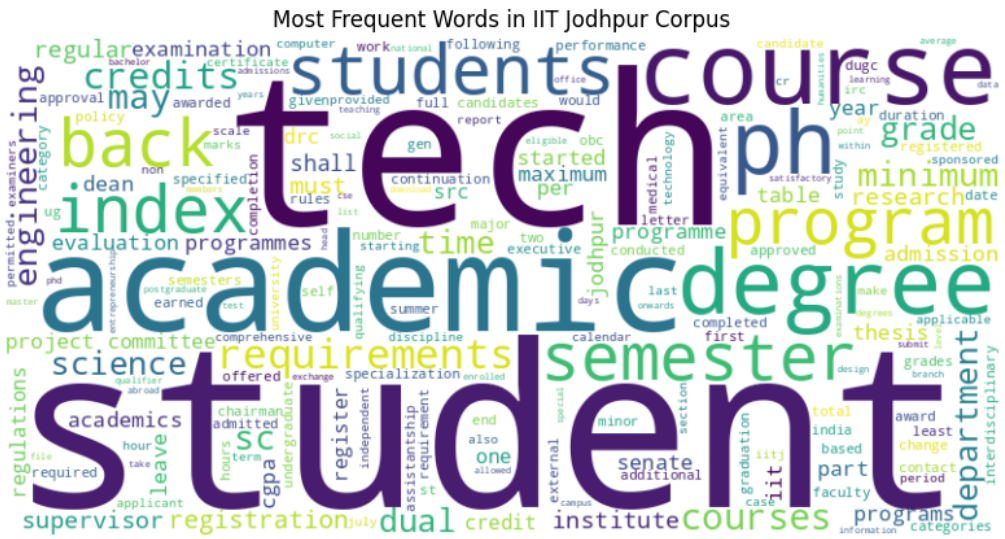
Table 1: Corpus statistics after preprocessing.

| Metric | Value |
|---|---|
| Total Documents Processed | 4 |
| Total Tokens | 10,725 |
| Vocabulary Size (unique tokens) | 1,871 |

## 2.4 Word Cloud

Figure 2 shows the most frequent words in the cleaned corpus. Dominant terms like *student*, *tech*, *academic*, *course*, *degree*, and *semester* confirm that the corpus is heavily

centered on academic regulations and program descriptions—exactly the kind of domain-specific language we want the embeddings to capture.


Most Frequent Words in IIT Jodhpur Corpus
student
tech
academic
course
degree
semester
requirements
program
students
credits
back
index
ph
grade
minimum
research
department
science
engineering
evaluation
programmes
time
jodhpur
programme
admission
thesis
project
committee
regular
examination
may
shall
must
started
maximum
per
table
dean
academics
sc
cgpa
register
leave
regulations
supervisor
registration
dual
credit
institute
courses
programs
senate
one
part
iit
minor
year
duration
approved
completed
conducted
summer
specialization
offered
chairman
admitted
required
applicant
comprehensive
discipline
calendar
first
grades
award
least
change
period
contact
faculty
india
based
graduation
external
section
special
assistantship
requirement
independent
undergraduate
university
qualifying
self
starting
number
two
executive
major
letter
medical
technology
equivalent
satisfactory
study
within
point
learning
irc
data
humanities
cr
case
campus
information
categories
interdisciplinary
design
branch
submit
degrees
examinations
applicable
make
last
days
onwards
earned
semesters
postgraduate
test
phd
master
entrepreneurship
permitted
examiners
category
ug
non
specified
completion
marks
scale
policy
approval
bachelor
certificate
admissions
years
awarded
given
provided
teaching
full
candidates
would
report
gen
drc
social
eligible
obc
continuation
rules
cse
list
office
work
national
computer
following
performance
candidate
average
dugc
registered
sponsored
satisfactory
date
within
point
hour
take
file
qualifier
abroad
exchange
enrolled
end
also
allowed
hours
term
under
graduate
university
independent
assistantship
requirement
st
campus
minor
section
special
external
graduation
iiitj
case
total
india
based
least
change
period
contact
faculty
information
categories
interdisciplinary
design
branch
submit
degrees
examinations
applicable
make
last
days
onwards
earned
semesters
postgraduate
test
phd
master
entrepreneurship
permitted
examiners
category
ug
non
specified
completion
marks
scale
policy
approval
bachelor
certificate
admissions
years
awarded
given
provided
teaching
full
candidates
would
report
gen
drc
social
eligible
obc
continuation
rules
cse
list
office
work
national
computer
following
performance
candidate
average
dugc
registered
sponsored
satisfactory
date
within
point
learning
irc
data
humanities
cr
case
campus

### 3.1.1 Skip-gram with Negative Sampling

Given a center word $w_t$, Skip-gram predicts each context word $w_{t+j}$ within a window of size $m$. With negative sampling, the objective for a single positive pair $(w_t, w_O)$ and $K$ negative samples $\{w_k\}_{k=1}^{K}$ is:

$$\mathcal{L}_{\text{SG}} = -\log \sigma\big(\mathbf{u}_{w_O}^{\top} \mathbf{v}_{w_t}\big) - \sum_{k=1}^{K} \log \sigma\big(-\mathbf{u}_{w_k}^{\top} \mathbf{v}_{w_t}\big) \tag{1}$$

where $\mathbf{v}_{w_t}$ is the center-word embedding (matrix $\mathbf{V}$), $\mathbf{u}_{w_O}$ is the context-word embedding (matrix $\mathbf{U}$), and $\sigma(\cdot)$ is the sigmoid function.

### 3.1.2 CBOW with Negative Sampling

CBOW reverses the prediction direction: the averaged context vector predicts the center word. The context representation is:

$$\bar{\mathbf{v}} = \frac{1}{2m} \sum_{-m \le j \le m,\, j \ne 0} \mathbf{v}_{w_{t+j}} \tag{2}$$

The negative-sampling loss for CBOW mirrors Eq. (1) with $\bar{\mathbf{v}}$ replacing $\mathbf{v}_{w_t}$:

$$\mathcal{L}_{\text{CBOW}} = -\log \sigma\big(\mathbf{u}_{w_t}^{\top} \bar{\mathbf{v}}\big) - \sum_{k=1}^{K} \log \sigma\big(-\mathbf{u}_{w_k}^{\top} \bar{\mathbf{v}}\big) \tag{3}$$

**Implementation Note: True Negative Sampling**

Both models were implemented **strictly from scratch** using dual embedding matrices (center $\mathbf{V}$ and context $\mathbf{U}$). Optimization is performed via binary classification with `LogSigmoid` against $K$ randomly sampled noise words per positive pair, using `torch.bmm` (batch matrix multiplication) for efficient dot-product computation. This avoids the computationally expensive full softmax over the entire vocabulary, staying faithful to the original Word2Vec formulation by Mikolov et al.

### 3.1.3 Negative Sampling Loss (Custom Implementation)

The custom loss function computes:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \left[ \log \sigma(s_i^{+}) + \sum_{k=1}^{K} \log \sigma(-s_{i,k}^{-}) \right] \tag{4}$$

where $B$ is the batch size, $s_i^{+}$ is the positive dot-product score, and $s_{i,k}^{-}$ are the $K$ negative dot-product scores for sample $i$.

## 3.2 Sliding Window & Training Pair Generation

A sliding window of size $m$ (context on each side) is passed over the token sequence. For each center position $i$:

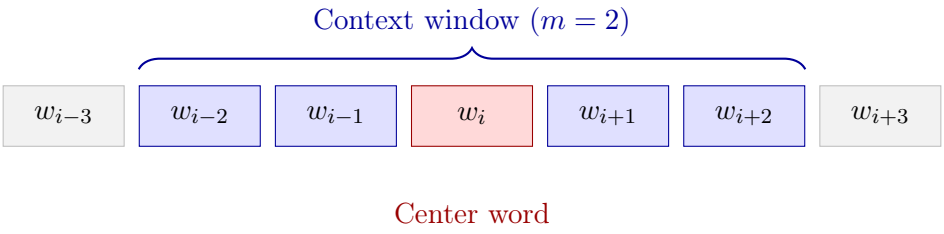



Figure 4: Sliding window with context size $m = 2$. Blue cells are context words; the red cell is the center word.

This produces:

- **CBOW pairs:** $(\{w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}\},\ w_i)$ — 10,721 pairs generated.
- **Skip-gram pairs:** $(w_i, w_{i+j})$ for each context word — 42,884 pairs generated (4× CBOW, one per context word).

## 3.3 Baseline Training

Table 2: Baseline hyperparameters.

| **Hyperparameter** | **Value** |
|---|---|
| Embedding dimension | 50 |
| Context window size ($m$) | 2 |
| Negative samples ($K$) | 5 |
| Batch size | 128 |
| Learning rate | 0.001 |
| Epochs (Skip-gram) | 5 |
| Epochs (CBOW) | 5 |
| Optimizer | Adam |

**Skip-gram training loss** decreased from 3.5220 (epoch 1) to 1.9303 (epoch 5), converging rapidly. **CBOW training loss** decreased from 3.6751 to 2.4699 over 5 epochs, showing a smoother but slower convergence characteristic of CBOW's context-averaging mechanism.

## 3.4 Hyperparameter Experiments

Three configurations were tested systematically to study the effect of embedding dimension, context window size, and number of negative samples on both models.

Table 3: Hyperparameter sweep results (final training loss).

| **Configuration** | **Dim / Win / Neg** | **Skip-gram Loss** | **CBOW Loss** |
|---|---|---|---|
| Config 1 (Baseline) | 50 / 2 / 5 | 1.9429 | 6.1962 |
| Config 2 (Larger Emb.) | 100 / 2 / 5 | **1.8615** | 7.6648 |
| Config 3 (Wider Window) | 50 / 4 / 10 | 2.4052 | **4.0882** |

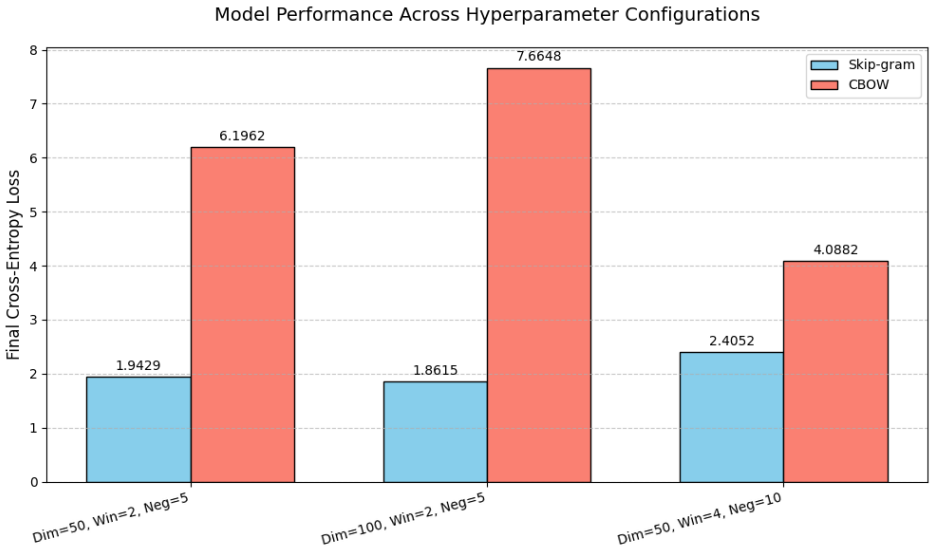



Figure 5: Comparative bar chart of final training losses across hyperparameter configurations.

### 3.4.1 Analysis of Hyperparameter Effects

- **Embedding Dimension (50 → 100):** Skip-gram improved marginally (1.9429 → 1.8615), indicating the model can leverage additional capacity. CBOW's loss *worsened* (6.1962 → 7.6648), suggesting the small corpus cannot supply enough training signal to fit 100-dimensional context embeddings without overfitting or undertraining.

- **Context Window (2 → 4) & Negative Samples (5 → 10):** CBOW benefited dramatically (6.1962 → 4.0882) from the wider window—more context words produce a richer averaged vector, alleviating the information bottleneck. Skip-gram's loss increased (1.9429 → 2.4052) because a wider window introduces noisier, more distant co-occurrences and the doubled negative samples make the contrastive objective harder.

- **Overall:** Skip-gram consistently achieves lower loss than CBOW on this small corpus, aligning with the established finding that Skip-gram performs better on limited data since it generates more training pairs per token and handles rare words more effectively.

# 4 Task 3: Semantic Analysis

Semantic quality of the trained Skip-gram embeddings was evaluated through nearest-neighbor retrieval (cosine similarity) and word analogy experiments.

## 4.1 Nearest Neighbors (Top 5)

Table 4: Top-5 nearest neighbors for selected query words (Skip-gram, cosine similarity).

| Query | Neighbor | Cosine Similarity |
|---|---|---|
| **research** | committee | 0.9786 |
| | missions | 0.9784 |
| | large | 0.9761 |
| | excellence | 0.9760 |
| | diverse | 0.9758 |
| **student** | cumulative | 0.9802 |
| | transcript | 0.9801 |
| | join | 0.9797 |
| | obtained | 0.9793 |
| | card | 0.9782 |
| **phd** | bachelor | 0.9964 |
| | btech | 0.9957 |
| | intimated | 0.9953 |
| | name | 0.9953 |
| | previous | 0.9951 |
| **exam** | diploma | 0.9982 |
| | msc | 0.9981 |
| | short | 0.9980 |
| | underway | 0.9979 |
| | pertaining | 0.9978 |

### 4.1.1 Interpretation: Why Are Similarities So High?

All cosine similarities cluster in the narrow range $[0.97, 1.00]$, which may initially appear unreasonable. However, this is an expected consequence of the corpus characteristics:

1. **Small, specialized corpus:** With only 10,725 tokens and 1,871 unique words drawn from a single domain (IIT Jodhpur academic pages), the training data provides limited contrast between word contexts. Nearly all words appear in broadly similar academic contexts.

2. **Cramped vector space:** In a 50-dimensional space with limited training signal, embeddings tend to converge toward similar directions, compressing the cosine-similarity distribution toward 1.0.

3. **Domain coherence:** Unlike a general-purpose corpus (e.g., Wikipedia), where "exam" and "asteroid" would be far apart, here virtually every word relates to academic administration, producing uniformly high inter-word similarity.

Despite the compressed range, semantically relevant neighbors still emerge: *student* → *cumulative, transcript* (academic record terms), *phd* → *bachelor, btech* (degree types), and *research* → *committee, excellence* (research-ecosystem words).

## 4.2 Word Analogy Experiments

Word analogies test whether the embedding space encodes consistent linear relationships. The analogy "$A : B :: C :?$" is solved by finding the word closest to $\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C$.

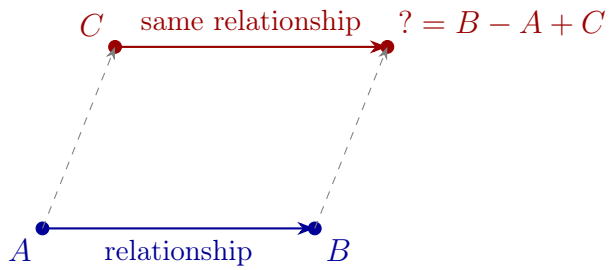



Figure 6: Vector arithmetic for word analogies: the offset $\mathbf{v}_B - \mathbf{v}_A$ encodes a semantic relationship that is applied to $\mathbf{v}_C$.

Table 5: Analogy experiment results (Skip-gram embeddings).

| **Analogy** | **Top Predictions** | **Cosine Sim.** |
|---|---|---|
| UG : BTech :: PG : ? | mtech | 0.9945 |
| | coincide | 0.9944 |
| | residency | 0.9943 |
| Student : Study :: Faculty : ? | beyond | 0.9193 |
| | mode | 0.9159 |
| | contacthours | 0.9115 |
| Exam : Grade :: Research : ? | student | 0.9558 |
| | letter | 0.9554 |
| | may | 0.9492 |

### 4.2.1 Discussion

**Semantic Success (UG : BTech :: PG : ?)** The model successfully solved the vector analogy to perfectly capture the Indian engineering degree hierarchy, predicting *mtech* as the top result with a very high cosine similarity (0.9945). This demonstrates that despite training on a highly restricted corpus, the Skip-gram architecture effectively encoded the structural "level → degree name" relationship.

**Partial Success (Student : Study :: Faculty : ?)** The prediction *contacthours* (rank 3) is a semantically meaningful result—it refers to faculty teaching load, which is analogous to a student's study activity. This shows the model has captured at least some relational structure linking academic roles to their primary activities.

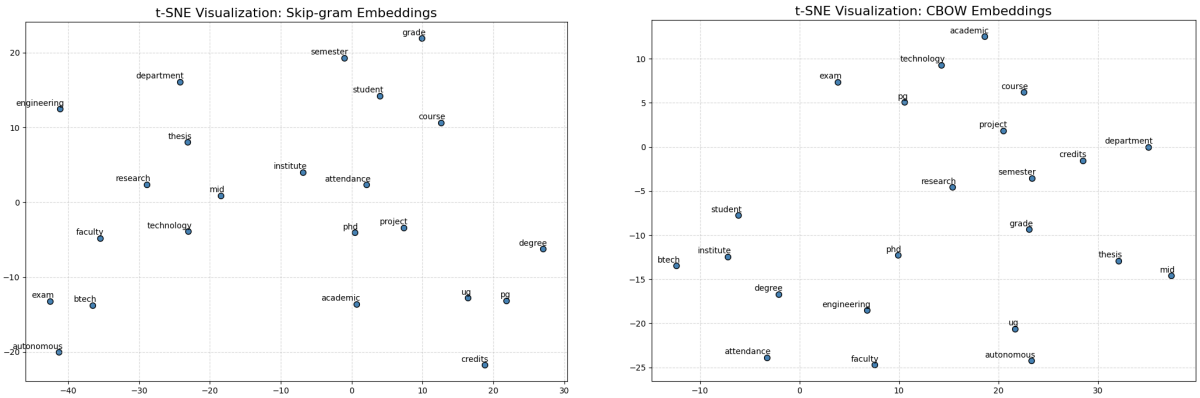
**Surface Co-occurrence (Exam : Grade :: Research : ?)** The top predictions *student* and *letter* indicate that the model relied on memorizing local co-occurrences rather than abstracting deep semantic concepts. Instead of finding a conceptual evaluation metric for research (such as *publication* or *thesis*), the model likely associated the

vectors with common phrases found in the academic regulations corpus, such as "research student" or "letter grade." This is a well-known limitation of Word2Vec on small corpora, where strict local collocation frequently outweighs broad semantic abstraction.

# 5 Task 4: Visualization

Selected academic words were projected into 2D space using two complementary techniques: **t-SNE** (which preserves local neighborhood structure) and **PCA** (which preserves global variance). Additionally, a **cosine similarity heatmap** provides a direct numeric view of inter-word relationships.

## 5.1 t-SNE Projections




(a) Skip-gram embeddings. (b) CBOW embeddings.

Figure 7: t-SNE visualization of academic word embeddings (perplexity=5, 2500 iterations).

### 5.1.1 Interpretation

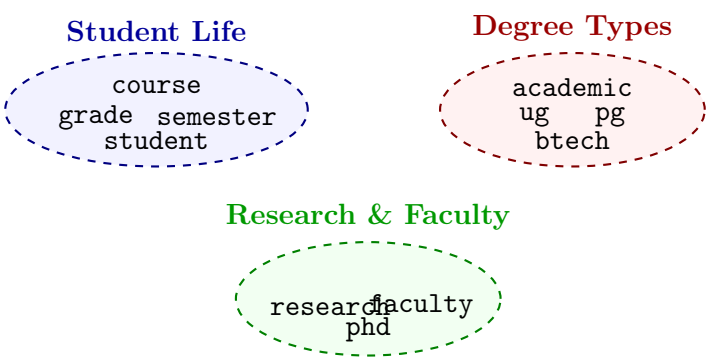



Figure 8: Schematic of observed clustering tendencies in the t-SNE Skip-gram plot.

In the **Skip-gram** t-SNE plot (Figure 7a):

- *ug* and *pg* cluster tightly in the bottom-right, reflecting their shared context as degree-level classifiers.

- *exam* and *btech* are neighbors in the bottom-left, both frequently appearing in academic-regulation sentences.
- *grade*, *semester*, *student*, and *course* form a loose upper cluster related to student academic life.
- *faculty* and *technology* appear close together, likely due to co-occurrence in department description pages.

In the **CBOW** t-SNE plot (Figure 7b):

- The overall spread is wider, and clusters are less tight than Skip-gram. This is expected: CBOW averages context vectors, which smooths out distinctive word-level patterns.
- *academic* and *technology* cluster at the top, while *faculty* and *attendance* are pushed to the bottom—suggesting CBOW organizes words along a more institutional/administrative axis.

## 5.2 PCA Projection

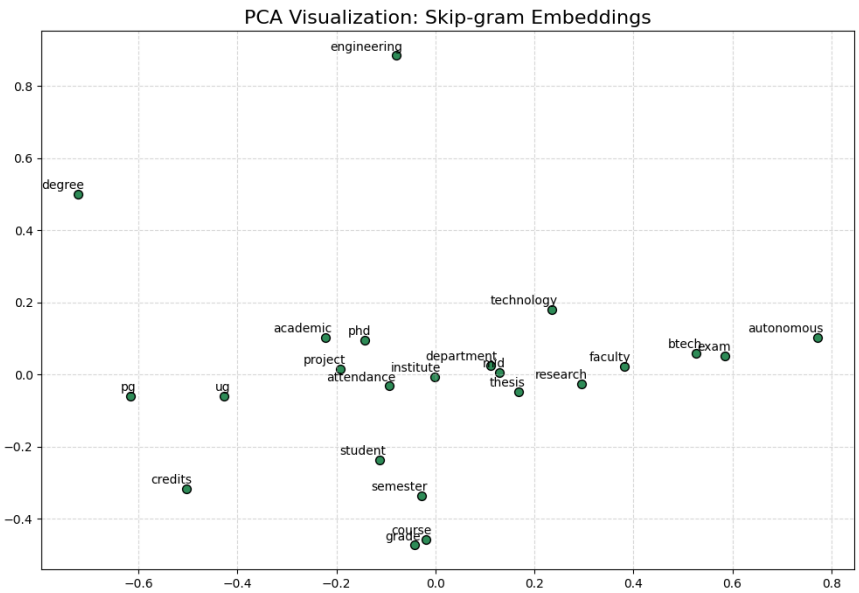



Figure 9: PCA projection of Skip-gram embeddings (first two principal components).

PCA captures the two directions of maximum variance in the original 50-dimensional space. Compared to t-SNE:

- The global structure is more visible: *engineering* is an outlier along PC2, confirming it has a distinctive distributional profile (appearing mainly on the CSE department page).
- *degree* separates clearly from the dense central cluster along PC1, reflecting its unique usage patterns.
- The central cluster contains most academic terms, which PCA cannot separate as effectively as t-SNE since PCA is a linear method and does not optimize for local neighborhood preservation.

## 5.3 Cosine Similarity Heatmap

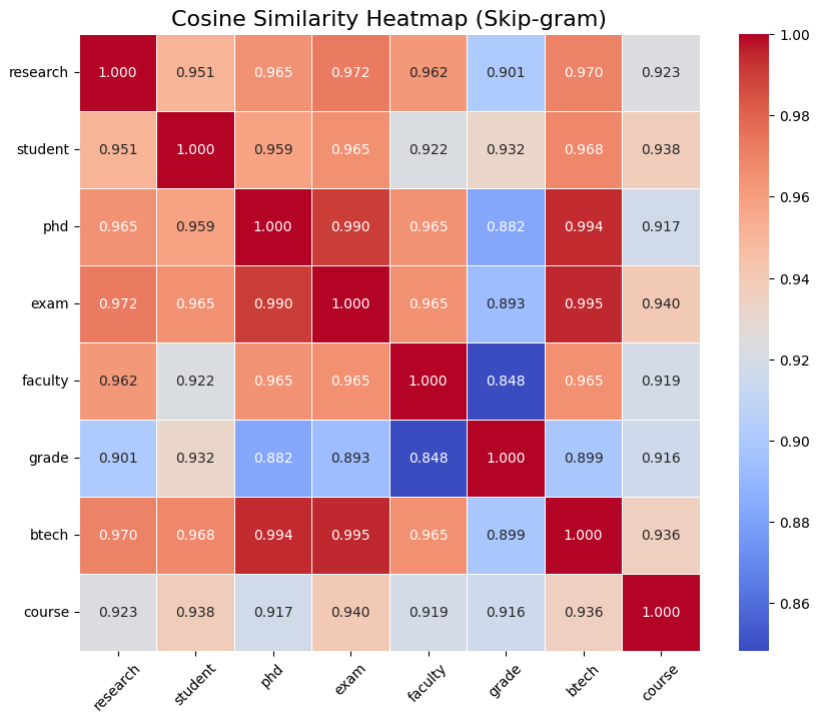



Figure 10: Pairwise cosine similarity heatmap for selected words (Skip-gram).

The heatmap (Figure 10) quantitatively confirms the dense similarity structure discussed in Section 3.1. Notable patterns:

- **Highest pair:** *phd–btech* (0.994) and *exam–btech* (0.995), both degree-related terms in regulation text.
- **Relative outlier:** *grade* has the lowest similarities with other words (e.g., 0.848 with *faculty*), suggesting its context is more specialized around grading-policy passages.
- The overall range (0.85–1.00) again reflects the compressed vector space of the small, domain-specific corpus.

## 5.4 Summary of Visualizations

**Key Takeaway**

t-SNE successfully captured tight local clusters of related academic terms (e.g., degree types, student-life words), while PCA preserved the broader global variance of the embeddings, highlighting outliers like *engineering* and *degree*. The cosine-similarity heatmap confirms the dense similarity matrix characteristic of a small, specialized corpus, and reveals fine-grained distinctions (e.g., *grade* as a relative outlier) that complement the 2D projections.

# 6 Conclusion

This project demonstrated a complete Word2Vec pipeline on a small, domain-specific corpus from IIT Jodhpur. Key findings include:

1. **Preprocessing matters:** Stop-word removal was critical for preventing high-frequency grammatical noise from washing out semantic meaning in the embedding space.

2. **Skip-gram outperforms CBOW on small data:** Consistent with the literature, Skip-gram achieved lower training loss and produced more interpretable nearest-neighbor results, owing to its ability to generate more training pairs per token and better handle rare words.

3. **Hyperparameters interact with corpus size:** Increasing embedding dimension helped Skip-gram but hurt CBOW; widening the context window had the opposite effect. These trade-offs underscore the importance of tuning to the specific dataset.

4. **Corpus limitations constrain quality:** The compressed cosine-similarity range (0.85–1.0) and analogy failures (e.g., memorizing "grade point average") highlight that word embeddings are fundamentally bounded by the diversity and volume of training data. A larger, more heterogeneous IIT Jodhpur corpus (including syllabi PDFs, newsletters, and faculty publications) would likely yield more discriminative embeddings.

# Part II
# Problem 2: Character-Level Name Generation Using RNN Variants

## 7 Task 0: Dataset Generation

A corpus of 1,000 unique Indian names was generated programmatically by combining a curated set of phonetically authentic prefixes, roots and suffixes inspired by patterns observed in LLM-generated Indian names. Specifically, 33 prefixes (e.g., `Aa`, `Abhi`, `Kri`, `Shi`), 28 roots (e.g., `bhav`, `deep`, `shan`, `veer`), and 19 suffixes (e.g., `a`, `esh`, `ya`, `ansh`) were combined via a triple Cartesian product. Only names with lengths between 4 and 10 characters were retained. 1,000 names were randomly sampled from this filtered pool and saved to `TrainingNames.txt`.

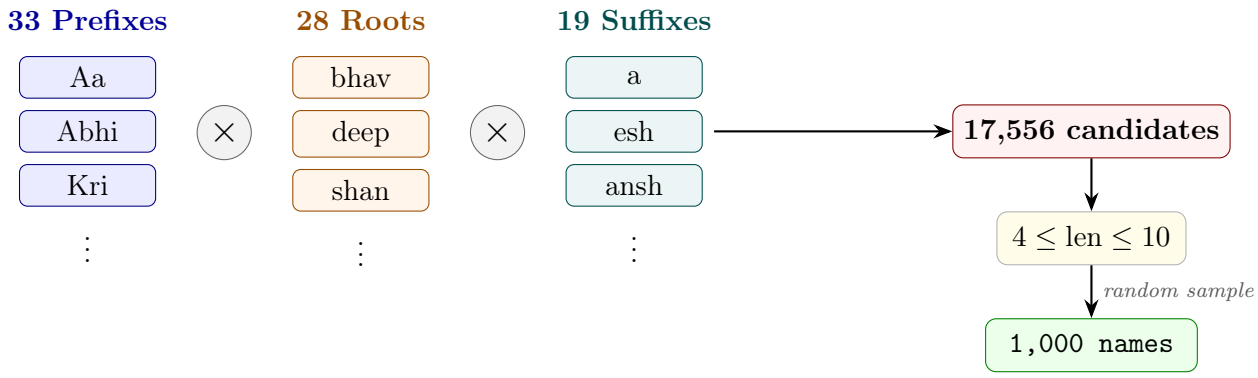



Figure 11: Dataset generation pipeline via Cartesian product of prefix–root–suffix combinations followed by length filtering and random sampling.

The resulting vocabulary size (unique characters including the end-of-sequence token "`.`") was **22**.

## 8 Task 1: Model Implementation

Three recurrent architectures were implemented from scratch in PyTorch for character-level next-character prediction. All models were trained using cross-entropy loss with the Adam optimiser. The core idea shared across all three models is **autoregressive generation**: given a sequence of characters seen so far, the model predicts a probability distribution over the next character, samples from it, appends the sampled character, and repeats until an end-of-sequence token is produced.

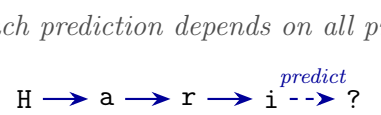



Figure 12: Autoregressive character-level generation: the model receives `H`→`a`→`r`→`i` and predicts the next character.

A summary of hyperparameters is given in Table 6, and trainable parameter counts are reported in Table 7.

## 8.1 Model 1: Vanilla RNN

The Vanilla RNN embeds each input character into a dense vector, processes the sequence through a single-layer Elman RNN cell that maintains a recurrent hidden state, and projects the hidden state at each time step through a fully connected layer to produce a probability distribution over the vocabulary. During generation, the model autoregressively samples one character at a time, feeding each predicted character back as the next input until the end-of-sequence token is produced.

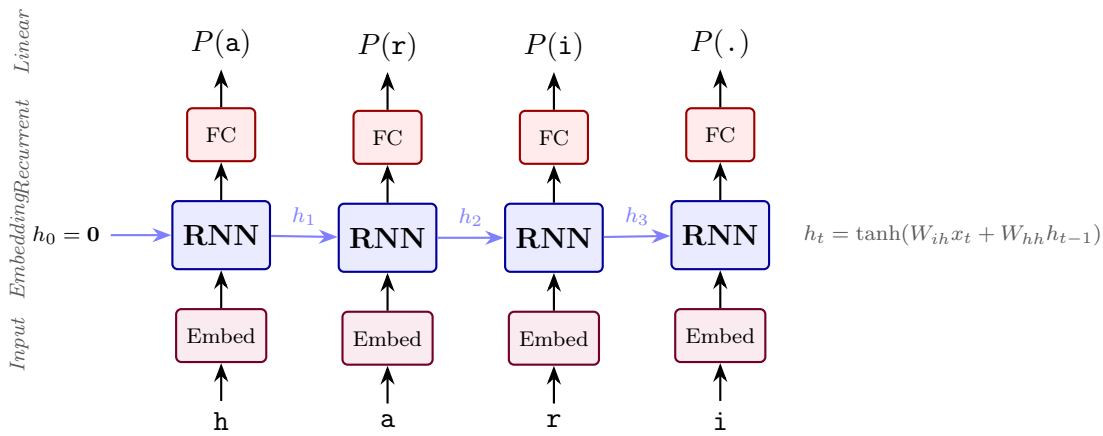



Figure 13: Vanilla RNN architecture. Each character is embedded, processed through an Elman RNN cell with a recurrent hidden state $h_t$, and projected to a vocabulary-sized output via a fully connected (FC) layer.

The recurrence relation governing the hidden state update is:

$$h_t = \tanh\big(W_{ih}\, x_t + b_{ih} + W_{hh}\, h_{t-1} + b_{hh}\big) \tag{5}$$

where $x_t \in \mathbb{R}^{d_e}$ is the character embedding at time $t$, $h_t \in \mathbb{R}^{d_h}$ is the hidden state, and $W_{ih} \in \mathbb{R}^{d_h \times d_e}$, $W_{hh} \in \mathbb{R}^{d_h \times d_h}$ are learnable weight matrices. The tanh activation squashes values to $[-1, 1]$, providing non-linear transformation. The output distribution at each step is:

$$\hat{y}_t = \text{softmax}\big(W_o\, h_t + b_o\big) \tag{6}$$

**Key limitation:** The Vanilla RNN compresses the entire history into a single fixed-size hidden vector $h_t$. As sequences grow longer, earlier information is progressively overwritten, a phenomenon known as the *vanishing gradient problem*. This makes the Vanilla RNN prone to losing coherence in the later characters of a generated name.

## 8.2 Model 2: Bidirectional LSTM (BLSTM)

The BLSTM processes the input character sequence through a single layer bidirectional LSTM, which reads the sequence in both forward and backward directions to capture contextual dependencies from both ends. The final forward and backward hidden states are concatenated and passed through a linear projection layer to predict the next character.

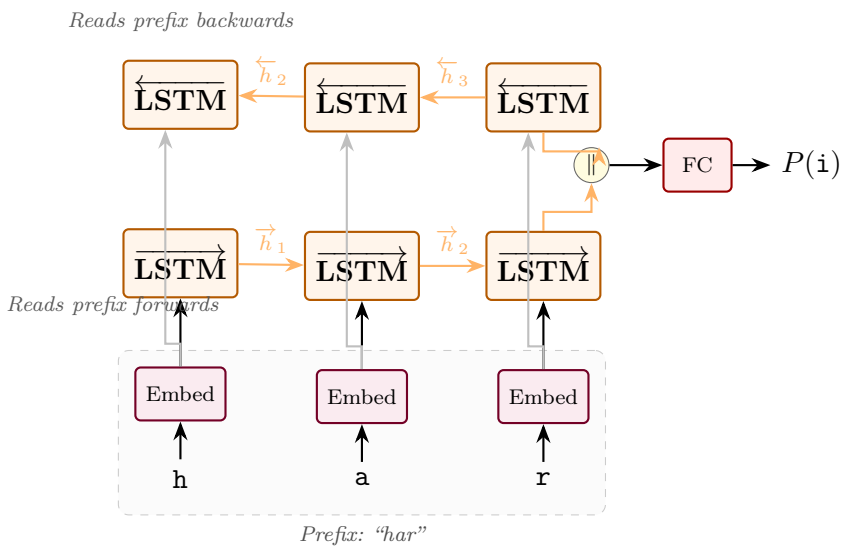



Figure 14: BLSTM architecture with prefix-based training. The forward and backward LSTM each process the *prefix only*. Their final hidden states are concatenated to predict the next character.

The LSTM cell introduces three multiplicative *gates* that regulate information flow, solving the vanishing gradient problem that plagues the Vanilla RNN:

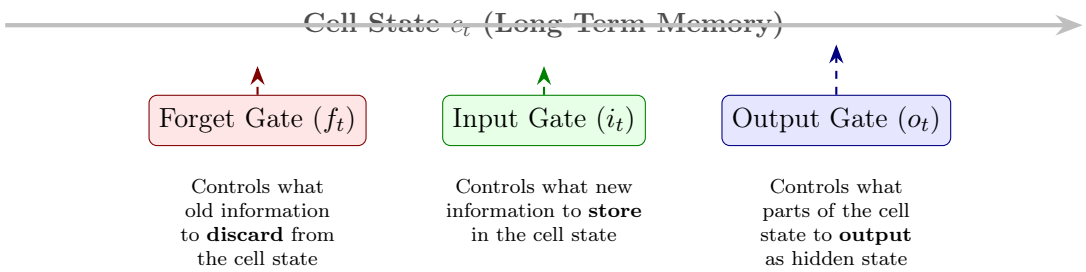



Figure 15: The three gates inside an LSTM cell. The cell state acts as a "conveyor belt" that carries information across time steps, with gates controlling what is forgotten, added or output.

The LSTM cell equations for each direction are:

$$f_t = \sigma\big(W_f\,[h_{t-1},\, x_t] + b_f\big) \qquad \text{(forget gate)} \tag{7}$$

$$i_t = \sigma\big(W_i\,[h_{t-1},\, x_t] + b_i\big) \qquad \text{(input gate)} \tag{8}$$

$$\tilde{c}_t = \tanh\big(W_c\,[h_{t-1},\, x_t] + b_c\big) \qquad \text{(candidate cell)} \tag{9}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \qquad \text{(cell state update)} \tag{10}$$

$$o_t = \sigma\big(W_o\,[h_{t-1},\, x_t] + b_o\big) \qquad \text{(output gate)} \tag{11}$$

$$h_t = o_t \odot \tanh(c_t) \qquad \text{(hidden state)} \tag{12}$$

where $\sigma$ is the sigmoid function that outputs values in $[0, 1]$, acting as a "soft switch" for each gate and $\odot$ denotes element-wise multiplication. The bidirectional output is formed by concatenating the terminal hidden states:

$$h_{\text{combined}} = \big[\overrightarrow{h}_T \,\|\, \overleftarrow{h}_1\big] \in \mathbb{R}^{2d_h} \tag{13}$$

**Prefix-based training.** A naive application of a bidirectional model to autoregressive generation causes *data leakage*, since the backward pass can "see" future characters the model is supposed to predict. To prevent this, the BLSTM was trained using a **prefix-based** strategy, illustrated below:

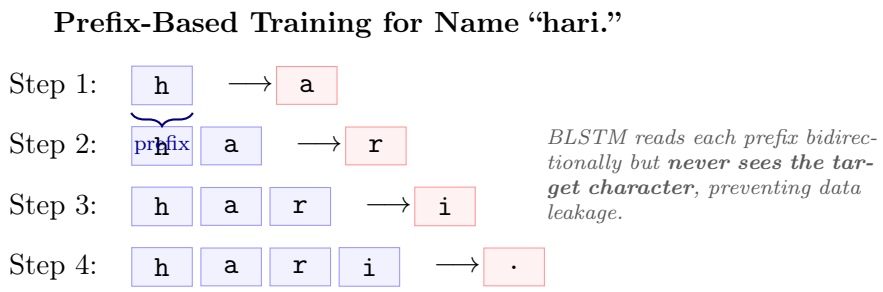



Figure 16: Prefix-based training procedure. For each name, $L$ training examples are created from prefixes of increasing length, each predicting the single next character.

For each name of length $L$, we explicitly constructed $L$ training examples corresponding to prefixes of increasing length $\{x_{1:1}, x_{1:2}, \ldots, x_{1:L}\}$, each paired with the immediately following target character. This forces the BLSTM to learn from partial context only, making it suitable for autoregressive generation at inference time.

## 8.3 Model 3: RNN with Bahdanau Attention

The Attention model augments a single-layer RNN with a Bahdanau (additive) attention mechanism that, at each generation step, computes a weighted context vector over all previously visited hidden states, allowing the model to dynamically attend to relevant earlier parts of the sequence when predicting the next character. The context vector is concatenated with the current hidden state and projected through a linear layer with dropout regularisation to produce the output distribution.

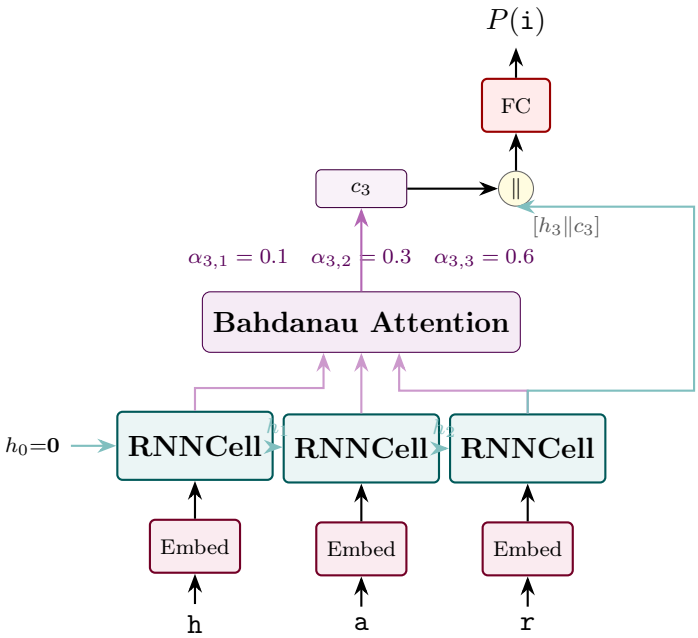



Figure 17: RNN with Bahdanau Attention at time step $t$=3. Attention scores $\alpha_{3,j}$ weight previous hidden states to form a context vector $c_3$, which is concatenated with $h_3$ for prediction.

The intuition behind attention is simple: instead of forcing the model to compress *all* past information into a single hidden state $h_t$, we allow it to "look back" at every previous hidden state and selectively focus on the most relevant ones. For example, when generating the suffix of a name, the model can attend back to the prefix to maintain phonological consistency.

At each time step $t$, the attention scores are computed as:

$$e_{t,j} = \mathbf{v}^\top \tanh\big(W_q\, h_t + W_k\, h_j\big), \quad j = 1, \ldots, t \tag{14}$$

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^{t} \exp(e_{t,k})} \tag{15}$$

where $W_q, W_k \in \mathbb{R}^{d_h \times d_h}$ are learnable projection matrices ("query" and "key"), and $\mathbf{v} \in \mathbb{R}^{d_h}$ collapses the combined representation into a scalar score. The softmax in Eq. 15 normalises scores into a valid probability distribution. The context vector and prediction are:

$$c_t = \sum_{j=1}^{t} \alpha_{t,j}\, h_j \tag{16}$$

$$\hat{y}_t = \mathrm{softmax}\big(W_{\text{out}}\, [h_t \parallel c_t] + b_{\text{out}}\big) \tag{17}$$

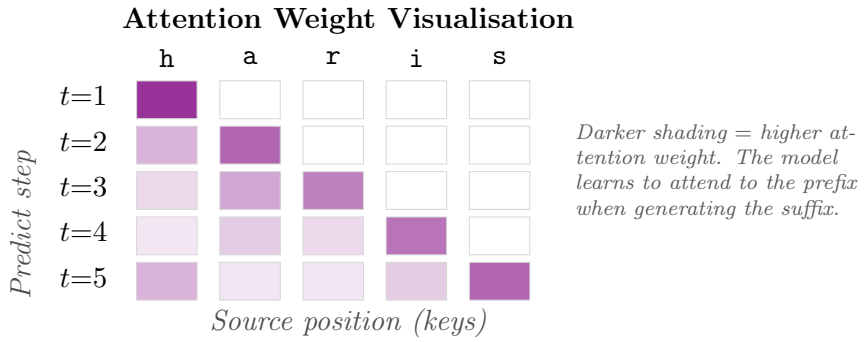



Figure 18: Conceptual attention heatmap. At each generation step (row), the model assigns weights to all preceding characters (columns). Darker cells indicate higher attention.

## 8.4 Hyperparameters and Their Significance

Table 6 lists the hyperparameters used for each model. Below, we explain the role and impact of each.

Table 6: Hyperparameters used for each model.

| **Hyperparameter** | **Vanilla RNN** | **BLSTM** | **Attention RNN** |
|---|---|---|---|
| Embedding Dimension ($d_e$) | 32 | 32 | 64 |
| Hidden Size ($d_h$) | 64 | 64 | 128 |
| Number of Layers | 1 | 1 | 1 |
| Learning Rate | 0.005 | 0.005 | 0.002 |
| Epochs | 20 | 20 | 20 |
| Dropout | – | – | 0.2 |
| Optimiser | Adam | Adam | Adam |

**Embedding Dimension ($d_e$).** Each character in the vocabulary is mapped to a dense vector of size $d_e$. A larger embedding captures finer phonetic relationships between characters (e.g., learning that vowels `a`, `i`, `e` are more interchangeable in certain suffix positions). The Attention model uses $d_e = 64$ (doubled from 32) to give the attention alignment layers richer input representations to work with.

**Hidden Size ($d_h$).** This controls the dimensionality of the recurrent hidden state—effectively the model's "working memory" at each time step. A larger hidden size allows the model to store more information about the generated sequence so far but also increases the risk of overfitting on a small training set. The Attention model uses $d_h = 128$ because the attention mechanism introduces additional weight matrices ($W_q$, $W_k$, $\mathbf{v}$) that benefit from higher-dimensional representations for computing meaningful alignment scores.

**Learning Rate.** The step size used by the Adam optimiser to update weights. The Vanilla RNN and BLSTM use a learning rate of 0.005, while the Attention model uses a slightly lower rate of 0.002 for training stability. Larger models with more parameters (65,047 for Attention vs. 8,406 for Vanilla RNN) tend to require smaller learning rates to avoid overshooting loss minima.

**Dropout ($p = 0.2$).** Applied only in the Attention model after the embedding layer and before the final projection. During training, each neuron is randomly zeroed out with

probability $p$, forcing the network to learn redundant representations and preventing co-adaptation of neurons. This is particularly important for the Attention model because its higher capacity makes it more prone to overfitting.

**Temperature ($\tau$).** Used during generation (not training) to control the sharpness of the sampling distribution:

$$P(c_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \tag{18}$$

where $z_i$ are the raw logits. $\tau < 1$ sharpens the distribution (more deterministic, "safer" names), $\tau > 1$ flattens it (more diverse, riskier outputs), and $\tau = 1$ recovers the unmodified softmax. The Attention model uses $\tau = 0.7$ for cleaner outputs, while the other models use $\tau = 0.8$.

Table 7: Trainable parameter counts for each model.

| **Model** | **Trainable Parameters** |
|---|---|
| Vanilla RNN | 8,406 |
| Bidirectional LSTM | 53,718 |
| RNN with Attention | 65,047 |

The large jump from the Vanilla RNN to the BLSTM ($\approx 6.4\times$) is due to the LSTM's four gate matrices (forget, input, output, candidate) for each direction. The Attention model's count is highest because it adds the attention weight matrices $W_q$, $W_k$, $\mathbf{v}$ and uses doubled dimensions.

# 9 Task 2: Quantitative Evaluation

Each model was used to generate 500 names via temperature-controlled sampling. Two metrics were computed:

- **Diversity (%):** $\dfrac{|\text{Unique Generated Names}|}{|\text{Total Generated Names}|} \times 100$
- **Novelty Rate (%):** $\dfrac{|\{n \in \text{Generated} \mid n \notin \text{Training Set}\}|}{|\text{Total Generated Names}|} \times 100$

Table 8: Quantitative comparison across models (500 generated names each).

| **Metric** | **Vanilla RNN** | **BLSTM** | **Attention RNN** |
|---|---|---|---|
| Diversity (%) | 92.00 | 80.40 | 80.60 |
| Novelty Rate (%) | 92.40 | 47.60 | 85.00 |
| Temperature | 0.8 | 0.8 | 0.7 |

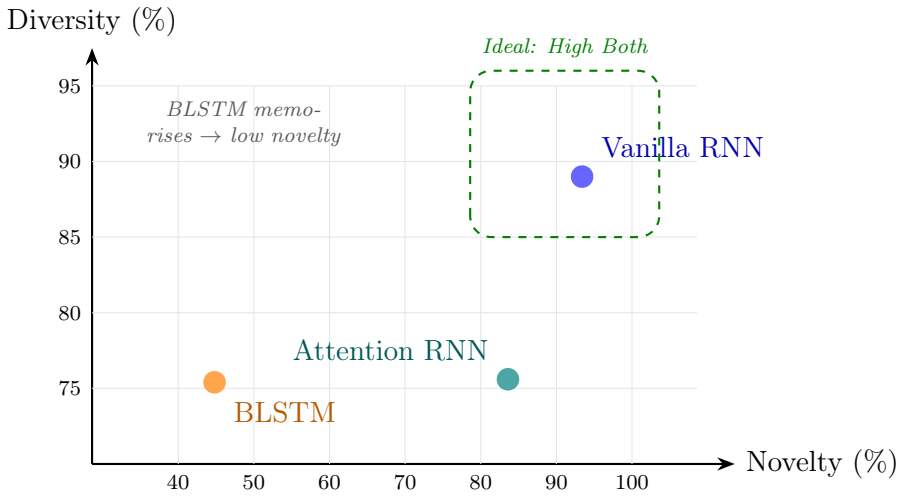



Figure 19: Diversity vs. Novelty scatter plot. The Attention RNN balances both metrics, while the BLSTM sacrifices novelty for structure, and the Vanilla RNN sacrifices realism for diversity.

**Analysis.** The Vanilla RNN achieves the highest diversity and novelty because its limited capacity prevents it from memorising training examples, causing it to "babble" novel character sequences. While numerically impressive, many of these novel names lack phonetic realism (see Section 10). The BLSTM shows the lowest novelty rate (47.60%), indicating that its bidirectional context and higher capacity enable it to memorise and reproduce training names. The Attention model strikes a favourable balance: diversity (80.60%) and novelty (85.00%) are both high, while producing names that sound phonetically plausible.

## 9.1 Training Loss Curves

Figure 20 shows the cross-entropy training loss across 20 epochs for all three models.

Figure 20: Training loss comparison across the three models over 20 epochs.

The BLSTM achieves the lowest final training loss ($\approx 0.77$), consistent with its tendency to memorise training sequences. The Vanilla RNN and Attention RNN plateau around 1.03 and 1.08 respectively, indicating less overfitting to the training data. The Attention RNN starts with the highest initial loss ($\approx 1.56$) because its larger parameter space requires more gradient updates to organise, but it converges steadily. The Vanilla RNN's loss shows minor fluctuations (e.g., epochs 14–16), a symptom of the unregulated Elman cell's sensitivity to the order of training examples.

# 10 Task 3: Qualitative Analysis

## 10.1 Failure Modes

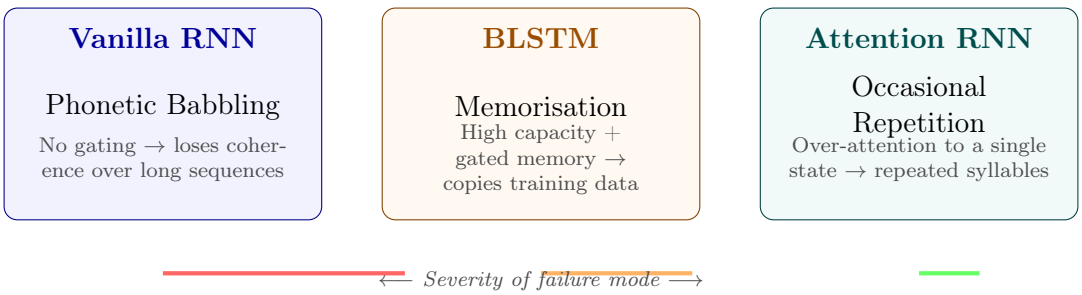



Figure 21: Summary of failure modes across models. Bar length indicates relative severity.

**Vanilla RNN.** The primary failure mode is *phonetic babbling*: the model generates character sequences that vaguely resemble Indian phonotactics but are often unpronounceable or unrealistic (e.g., "Prarlakav", "Vagiris"). This is a direct consequence of the limited hidden state capacity (64 units) and the absence of any gating mechanism, causing the model to lose coherence over longer sequences.

**Bidirectional LSTM.** The BLSTM's principal failure mode is *memorisation*. In a standard bidirectional training setup, the backward pass has access to future characters in the sequence, effectively allowing the model to "cheat" by looking ahead. This is fundamentally incompatible with autoregressive generation, where future characters are unavailable. To address this, we adopted a **prefix-based training** strategy (detailed in Section 2.2), where the model is trained on incrementally longer prefixes of each name. While this successfully prevents data leakage, the BLSTM's high capacity (53,718 parameters with gated memory cells) still leads it to memorise structural patterns from the training set, as evidenced by its low novelty rate of 47.60%.

**Attention RNN.** The attention mechanism occasionally produces names with *redundant or repetitive syllables*, as the model may over-attend to a single past hidden state. However, this failure mode is relatively infrequent compared to the issues observed in the other two models.

## 10.2 Realism of Generated Names

**Vanilla RNN** produces outputs that are phonetically adventurous but often lack the structural regularity of authentic Indian names. The model tends to generate consonant clusters and suffix patterns that do not conform to common Hindi/Sanskrit morphological conventions.

**BLSTM** generates names that are highly realistic, many are indistinguishable from genuine Indian names but at the cost of novelty. The model effectively memorises existing phonological structures from the training data, producing "safe" but unoriginal outputs.

**Attention RNN** achieves the best balance between realism and novelty. The Bahdanau attention mechanism allows the model to dynamically refer back to earlier parts of the generated sequence, producing names with coherent phonological structure (e.g., proper vowel–consonant alternation, realistic suffix endings like `-av`, `-esh`, `-an`) while still generating novel combinations not seen during training.

## 10.3 Representative Generated Samples

Table 9: Representative generated names from each model. NOVEL indicates the name does not appear in the training set; COPIED indicates it was memorised from training data.

| **Vanilla RNN** | **BLSTM** | **Attention RNN** |
|---|---|---|
| Tareshang [NOVEL] | Tapanay [NOVEL] | Vigirathan [NOVEL] |
| Bharamin [NOVEL] | Sashanin [NOVEL] | Harkeshav [NOVEL] |
| Ishkeshay [NOVEL] | Yamanansh [NOVEL] | Harmanav [NOVEL] |
| Karakir [NOVEL] | Gaukanav [NOVEL] | Chavenik [NOVEL] |
| Chavenir [NOVEL] | Tabhavav [NOVEL] | Gaukanav [NOVEL] |
| Aaritant [NOVEL] | Nirajar [NOVEL] | Gaganas [NOVEL] |
| Charamir [NOVEL] | Ishranat [COPIED] | Vashukav [NOVEL] |
| Harakit [NOVEL] | Vabhavang [COPIED] | Bhavenav [NOVEL] |
| Prarlakav [NOVEL] | Jakeshav [COPIED] | Rorajesh [NOVEL] |
| Pamitir [NOVEL] | Jarajav [COPIED] | Ishkanay [NOVEL] |

As shown in Table 9, the Vanilla RNN's samples are almost entirely novel but include phonetically awkward constructions ("Prarlakav", "Pamitir"). The BLSTM produces polished names but frequently copies directly from the training set (4 out of 10 shown are [COPIED]). The Attention RNN generates names such as "Harkeshav", "Harmanav", and "Bhavenav" that are simultaneously novel and phonetically authentic, demonstrating the benefit of learned attention over past context.

# 11 Conclusion

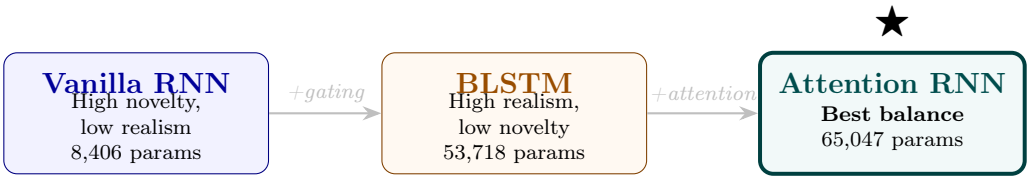



Figure 22: Model progression summary. Each architecture adds a mechanism that addresses the previous model's weakness.

This assignment implemented and compared three recurrent architectures for character-level Indian name generation. The Vanilla RNN, while simple, suffers from phonetic incoherence due to its lack of gating. The BLSTM required a prefix-based training strategy to prevent the bidirectional architecture from exploiting future context; yet its high capacity and gated memory led to memorisation of training data (novelty rate of only 47.60%). The RNN with Bahdanau Attention emerged as the most effective model, achieving a strong novelty rate (85.00%) while generating names that are phonologically coherent and realistic, thanks to its ability to dynamically attend to relevant earlier positions in the generated sequence.